Summary on Week 6 Mini Project

Section A: LLM Foundations & Hugging Face

1.Hugging Face Setup & Text Generation

```
Prompt: AI is transforming industries by

Three different continuations:

Continuation 1: AI is transforming industries by turning the focus on innovation into a business.
It’s a great step.


The most recent report from the European Commission on the evolution of the European economy (the
report is available on the website) confirms

Continuation 2: AI is transforming industries by bringing energy efficiency, energy efficiency
and innovation to areas of energy economy. With a team of 30 employees from six states and an
innovative energy sustainability program, we are transforming energy efficiency, energy
efficiency and innovation to areas of energy economy

Continuation 3: AI is transforming industries by reducing the time time it takes to fix all
barriers between them to innovation.




"One of the reasons the data center industry is creating new jobs is that all these technologies
are still being used as part
```

The output shows 3 different continuations of the given sentence. We have used Hugging face transformers with the model ‘distilgpt2’ to generate these texts.

‘Distilgpt2’ mainly provides texts as the continuation of previous texts. So we have provided the given text to create the continuations to it.

2.Tokenization Demo

```
Tokens: ['LL', 'Ms', 'Ġare', 'Ġpowerful', 'Ġtools', 'Ġfor', 'Ġnatural', 'Ġlanguage',
'Ġunderstanding', '.']
Token IDs: [3069, 10128, 389, 3665, 4899, 329, 3288, 3303, 4547, 13]
Sequence Length: 10
```

For the tokenization, we have used the provided sentence "LLMs are powerful tools for natural language understanding.” To create the tokens.

Tokenisation is nothing but converting human language (text) into numbers that a model can understand.

A sentence can be divided into smaller meaningful words or subwords called tokens as shown in the above picture.

The output Tokens contains G at the starting of each token because we have used ‘Distilgpt2’ model to create the tokens and G means space before the word in Distilgpt2.

TokenIDs are the numerical representations of tokens created using ‘tokenizer’ of Transformers.

Section B: Prompt Engineering - Summarization

```
=== Section B: Prompt Engineering ===

--- Summarization Task ---
Prompt: Summarize the following text in under 30 words:

Artificial intelligence (AI) refers to the simulati...
Generated Summary: Summarize the following text in under 30 words:

Artificial intelligence (AI) refers to the simulation of human intelligence in machines that are
programmed to think like humans and mimic their actions. The term may also be applied to any
machine that exhibits traits associated with a human mind such as learning and problem-solving.

Summary: AI
```

In this prompt Engineering Summary generation task, what I have observed is that, whatever the length we have mentioned in the prompt, the models only writes the sentences based on the length we have given in our code i.e, max_length=30.

When I have given max_length=150, it has generated the summary with few sentences, generated output repeated parts of the input text and included additional unrelated information instead of producing a concise summary eventhough when we mentioned to provide text under 30words, but when given max_lenght=30, it stopped at ‘AI’.

Even if we write a very clear prompt, the model only performs according the values we have provided in the code.

Q&A and Creative Text Generation

```
Generated Answer: Answer the following question:

Question: What is the capital of France?
Answer: The capital of France is actually the first capital in the world, its first-class capital.
When is French capital capital the fourth capital? How long

--- B3: Creative Text Generation ---
Generated Poem:
Write a short poem about artificial intelligence.
The poem must have exactly 4 lines.
Each line should be simple and meaningful.

AI minds so bright and clear,
Now, let me do this. I am going to be really careful here. You have the power to do anything I want
to. That's the way things should work. But when you want the best of humans, the more you can do
things that you're going for. My goal is to create a great poem. It's not about human creativity.
And it's about our ability to make the difference between ourselves
```

The Q&A and Creative Text generation parts was quiet frustrating because the outputs were not related to the prompts we are giving.

When we have given the question to answer the capital of France, it generated a long text but not the actual answer which is paris.

I have tried changing the max_length of the sentence to 10 thinking that would give me the correct result. But that's of no use as we are using the 'Distilgpt2' model.

Distilgpt2 model don't focus on strict instruction but focuses on the text generation and sentence continuation, it predicts the most probable next tokens rather than thinking about correctness.

It is very important to use the models that works by following the instructions on the given prompt rather than just focusing on sentence continuation.

Creative text

Creative text is also failed just because of using Distilgpt2 model. This model don't focus on strict instructions but focuses on text completion. So it generated the text without thinking of the poetic structure.
I came to know that it is very very important to use the perfect models that satisfy our needs.

Section c: Embeddings with Gensim- Word Embeddings

```
Word: 'king'
First 10 values of vector: [ 0.50451   0.68607  -0.59517  -0.022801  0.60046  -0.13498  -0.08813
  0.47377  -0.61798  -0.31012 ]
Top 5 similar words:
  prince          → 0.8236
  queen           → 0.7839
  ii              → 0.7746
  emperor         → 0.7736
  son             → 0.7667

Word: 'queen'
First 10 values of vector: [ 0.37854   1.8233   -1.2648   -0.1043    0.35829   0.60029  -0.17538
  0.83767  -0.056798 -0.75795 ]
Top 5 similar words:
  princess        → 0.8515
  lady            → 0.8051
  elizabeth       → 0.7873
  king            → 0.7839
  prince          → 0.7822

Word: 'diamond'
First 10 values of vector: [-0.4958   0.78421 -0.606    1.3967   0.28888 -0.2058  -0.10745
-0.33252
  1.3608   0.15091]
Top 5 similar words:
  gold            → 0.7715
  diamonds        → 0.7663
```

Embeddings are numerical representations of text or sentences that capture meaning. Embeddings convert words into vector(list of numbers) so that the computer can understand the relationships easily.

Gensim is a Python library used to understand text by converting words and documents into meaningful numbers (embeddings).

Genism is particularly used for Embeddings, similarity and vector search. I checks the similarity between words like when a word is given, it generates its related words as shown in the about output.

We have used the pretrained model "glove-wiki-gigaword-50" to create the glove vectors, the vector representations of words, When the most_similar() function is used, Gensim internally compares the vector of a given word with vectors of all other words in the vocabulary. Even though cosine similarity is not explicitly written in the code, it is used behind the scenes to measure how close the meanings of words are.

Based on this similarity calculation, Gensim returns the top five words that are most semantically similar. This is why words like *king* return related terms such as *queen* and *prince*, showing that embeddings successfully capture real-world relationships between words.

Sentence-Level Embeddings

```
Similarity Matrix:
 [[1.0000001  0.7538497  0.6955153  0.8183563  0.703131  ]
 [0.7538497  1.         0.76865256 0.71397036 0.73523057]
 [0.6955153  0.76865256 1.         0.802506   0.84309214]
 [0.8183563  0.71397036 0.802506   0.9999999  0.81852686]
 [0.703131   0.73523057 0.84309214 0.81852686 0.9999998 ]]

Most similar pair:
'Deep neural networks process information' ↔ 'Artificial intelligence transforms industries'
Score: 0.843
```

Here we have use Cosine_similarity explicitly to create word embeddings and create the similarity matrix.

Cosine similarity is a way to measure how similar two vectors are. Its basically used to find text similarity.

For every sentence each word is converted into vector and an average of those vectors was calculated so that for each sentence a vector is created. This is called sentence embedding.

Sentence embedding is nothing but a vector(numerical) representation of a sentence.

Once we have sentence embedding we can calculate the similarity between the sentences as similar vector values can be a result to similar sentences. This similarity is calculate using cosine similarity.

The similarity matrix for the sentences have been generated, the most similar sentences are displayed with the similarity score.

Section D: Application Exploration

Transformer Application

```
=== Hugging Face Sentiment Classification ===

Loading sentiment analysis model...
Analyzing sentiment for 10 custom inputs...

[{'label': 'POSITIVE', 'score': 0.9998775720596313}, {'label': 'NEGATIVE', 'score':
0.9997151494026184}, {'label': 'POSITIVE', 'score': 0.9998351335525513}, {'label': 'POSITIVE',
'score': 0.9965566396713257}, {'label': 'POSITIVE', 'score': 0.9998859167098999}, {'label':
'NEGATIVE', 'score': 0.9997983574867249}, {'label': 'POSITIVE', 'score': 0.9996623992919922},
{'label': 'POSITIVE', 'score': 0.9998675584793091}, {'label': 'NEGATIVE', 'score':
0.9991864562034607}, {'label': 'NEGATIVE', 'score': 0.9498637914657593}]
```

I have used the Hugging Face Transformer Pipeline to define the sentiment of the statements, I took 10 statements to check their sentiment.

We have used the ‘sentiment-analysis’ a special alias in Hugging face to check the sentiment of the statements.

We have used the predefined model “distilbert-base-uncased-finetuned-sst-2-english".

Hugging Face Transformers have made it easy to calculate the sentiment of sentences by just simply providing the predefined functions. We can easily summarise, translate, classify the sentences using the simple commands only. This makes the transformers very user friendly and easy to work with.

Key observations about LLMs vs. embeddings:

LLMs and Embeddings play a very different role while working with text. LLMs will create a new text every time we run while the embeddings will create the same numerical values for every run.
LLMs are basically used to generate text, write summaries, answer questions, write creative content like poems and stories where as Embeddings will represent the meaning of the sentences in numerical form known as vectors.

LLMs basically understand and generate the human like text by predicting the next word based on the context provided while Embeddings on the other hand don’t ever create a new

text, they just work on the given test. Embeddings generates the mathematical representations of words and used calculate the similarity between the sentences using the vector representations and cosine similarity.

LLMs help the machines to communicate the meaning while Embeddings help them to understand the meaning.

Applications of Vector Search & Embeddings:

1. **Semantic Search** : is used to find documents or content based on the context
2. **Recommendation Systems**: Suggest similar items
3. **Duplicate Detection :** Spot repeated content
4. **Chatbots & Virtual Assistants** → Retrieve relevant info before generating answers
5. **Sentiment Analysis** → Group feedback by meaning
6. **Fraud Detection** → Identify risky patterns
7. **Question-Answer Systems** → Quickly find answers in large datasets

This mini project has shown how LLM can generate different types of texts based on the given scenarios. It is much useful in Prompt Engineering but using the good model can result in correct outputs. Embeddings take the NLP to next level by using things like cosine similarity and calculating the similarity between the sentences. This makes it easy to compare the sentences or find related content efficiently. Together, LLMs and embeddings complement each other, enabling smarter and more flexible AI applications.